

# Chasing the Spread: A Differential Equations Approach to Crypto Pair Trading

Elvis Obondo

Differential Equations 230

May 13, 2026

# Contents

|   |                                                        |    |
|---|--------------------------------------------------------|----|
| 1 | Introduction: What Are We Even Trying To Do?           | 2  |
| 2 | Background: Why Do We Use Log Prices?                  | 2  |
| 3 | Finding the Pairs: Pearson or Spearman?                | 4  |
| 4 | The Model: So What Is The ODE?                         | 6  |
| 5 | Methods: How Do We Actually Solve It?                  | 8  |
| 6 | Model Iterations: Did It Work? (Spoiler: Not At First) | 10 |
| 7 | Results: So What Did I Actually Find?                  | 14 |
| 8 | Discussion: Where Does This Go From Here?              | 16 |

## Abstract

Can you model a cryptocurrency spread with a differential equation? Probably not perfectly, but this paper makes a serious attempt. Using hourly price data for ETH and LINK, I construct a system of ODEs built around mean reversion and an undamped market oscillator, solve it numerically with RK45, and watch it slowly get better with each iteration. The math is honest about its limitations: noise is real, parameter estimation is hard, and the market does not care about your ODE. But the underlying structure is there, and this paper is the first step toward finding it properly.

## 1. Introduction: What Are We Even Trying To Do?

Differential equations are types of equations used for modeling systems that change over time. Most applications taught are usually in physics or biology. This paper asks whether they belong in finance too, specifically whether the price spread between two correlated cryptocurrencies can be captured by a system of ODEs. Using ETH and LINK as my test case, I build, break, and rebuild a model until something resembling structure emerges.

## 2. Background: Why Do We Use Log Prices?

When it comes to financial data, I am tracking the movement of price across time. The natural way to think about it is the difference in raw price at different points in time. But raw price differences are misleading. Consider the following:

ETH goes from \$1000 to \$1010. LINK goes from \$10 to \$20. Same \$10 price movement, but completely different economic significance. ETH sees a 1% price increase and LINK sees a 100% price increase. So the next step seems to use percentage change instead right? Well not really

### Why Not Percentage Change?

Suppose ETH's price movements are as follows:  $\$1000 \rightarrow \$1010 \rightarrow \$1007.50$ , and LINK's movements are:  $\$10 \rightarrow \$20 \rightarrow \$15$ .

The percentage changes would be:

$$\text{ETH: } 1\% \rightarrow -0.25\%$$

$$\text{LINK: } 100\% \rightarrow -25\%$$

What is the total percentage change? You might be tempted to add them:

$$\text{ETH: } 1\% + (-0.25\%) = 0.75\%?$$

$$\text{LINK: } 100\% + (-25\%) = 75\%?$$

But the actual percentage change is:

$$\text{ETH: } 1.01 \times 0.9975 = 1.00748 \quad (0.748\% \text{ increase})$$

$$\text{LINK: } 2 \times 0.75 = 1.5 \quad (50\% \text{ increase})$$

Percentage change is multiplicative, not additive. This is manageable across three time periods but becomes a serious problem when working with millions of data points across different assets. (This was a problem of serious concern back in the day when math was handled more manually than it is today)

## So What Do We Use?

The fix is logarithms. Logarithms turn multiplication into addition:

$$\ln(a) + \ln(b) = \ln(a \cdot b) \quad \leftarrow \text{multiplication becomes addition}$$

$$e^{\ln(a \cdot b)} = a \cdot b \quad \leftarrow \text{exponentiate to recover the price ratio}$$

Carrying this over to the example:

$$\text{ETH: } \ln\left(\frac{1010}{1000}\right) + \ln\left(\frac{1007.50}{1010}\right) \approx 0.00748$$

$$\text{LINK: } \ln\left(\frac{20}{10}\right) + \ln\left(\frac{15}{20}\right) \approx 0.405$$

The log returns now correctly reflect the very different economic significance of each move. No messy multiplication across millions of time steps, just clean addition. From this point forward, all prices in this paper are log prices.

## 3. Finding the Pairs: Pearson or Spearman?

Before building a model, I need to identify which cryptocurrency pairs are actually correlated. That requires a correlation metric. But not all correlation metrics are created equal, and the choice matters more than it might seem.

### What Is Each Metric Actually Asking?

Pearson asks: is there a constant linear rate of change between these two assets? This means that when ETH moves 3%, LINK moves 5% **each and every time**. It assumes a constant linear proportional relationship, which is not the case in financial markets.

Spearman asks something different: if ETH had a big day relative to its history, did LINK also have a big day relative to its history? The exact magnitude of the move is irrel-

evant. Spearman focuses strictly on ranking. Did they have unusually good days together? Unusually bad days together?

## Why Does This Matter?

Consider three different market conditions:

- **Calm market:** ETH moves 0.8%, LINK moves 1%
- **Bull market:** ETH moves 20%, LINK moves 30%
- **Bear market:** ETH crashes 30%, LINK crashes 45%

Spearman still sees the correlation across all three conditions because it is tracking direction and order, not magnitude. Pearson struggles because there is no single linear relationship that accurately describes all three regimes at once. A crash day that is 10 times larger than a normal day gets 100 times the weight in the Pearson calculation because it squares the deviations. One extreme move can overpower hundreds of normal days of data.

This is also why the distribution of the data matters. Pearson is best suited for normally distributed data, the classic bell curve. Financial data, and crypto in particular, follows a Student- $t$  distribution with fat tails, meaning extreme outliers are far more likely than a normal distribution would suggest. Spearman handles this naturally since it only cares about rank.

## So Why Did I Use Pearson?

It depends on the type of ODE used. I chose a linear ODE that assumes linear relationships between the two assets, which means I need a correlation metric that is consistent with that assumption. Pearson does exactly that. Spearman, despite its robustness, would be measuring something my linear model cannot actually use.

To find my pair, I computed a Pearson correlation matrix across a basket of 33 cryptocurrencies using pandas' `df.corr()` function, masked the diagonal to avoid self-matches,

and selected the pair with the highest correlation value across the entire basket. That pair was ETH and LINK, with a Pearson correlation of 0.9448. The code used to identify this pair is shown in Figure 1.

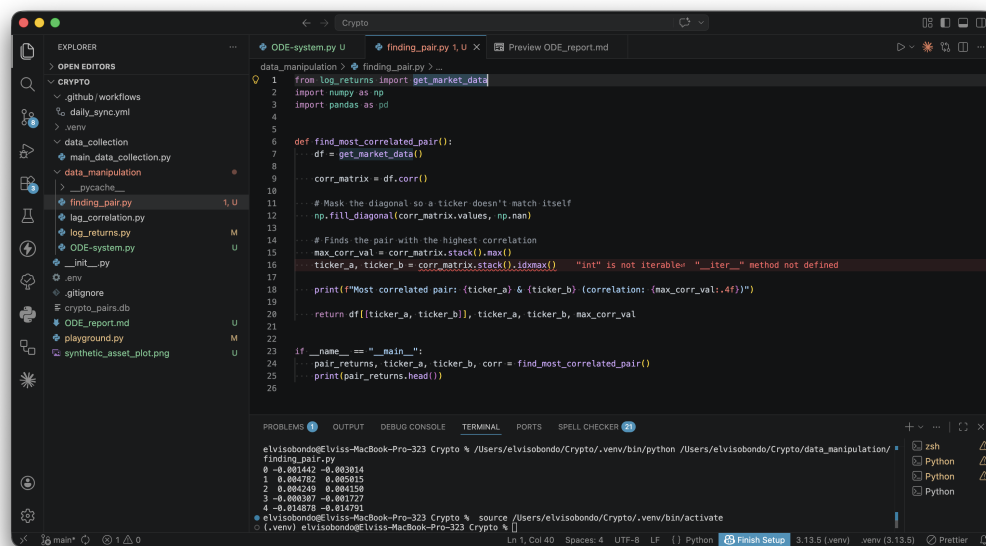


Figure 1: Python code used to compute the Pearson correlation matrix across 33 cryptocurrencies and identify the most correlated pair. ETH and LINK returned a correlation of 0.9448.

I treat the choice of Pearson as a deliberate simplifying assumption tied directly to my linear ODE framework, and I return to its limitations in the Discussion.

## 4. The Model: So What Is The ODE?

Now that I have my pair, I need a model. The core idea behind pair trading is mean reversion: when the spread between two correlated assets diverges, it tends to come back. The question is whether I can capture that behavior with a differential equation.

### The System

I model the prices of ETH and LINK as a coupled system of ODEs:

$$\frac{dX}{dt} = -k(X - Y) \quad (1)$$

$$\frac{dY}{dt} = -k(Y - X) \quad (2)$$

Where  $X$  is the log price of ETH,  $Y$  is the log price of LINK,  $t$  is time measured in 1-hour candles, and  $k$  is the coupling force between the two assets.

### What Is $k$ ?

$k$  is the parameter that controls how strongly the two assets pull toward each other. A large  $k$  means the spread closes quickly. A small  $k$  means the assets move more independently. Think of  $k$  as a measure of how tightly coupled ETH and LINK are in the market.

### Why Does This Make Sense?

Consider what each equation is saying. When  $X > Y$ , the term  $(X - Y)$  is positive, so  $\frac{dX}{dt}$  is negative, meaning ETH is being pulled down toward LINK. At the same time,  $(Y - X)$  is negative, so  $\frac{dY}{dt}$  is positive, meaning LINK is being pulled up toward ETH. The two assets are always pulling toward each other at a rate proportional to how far apart they are. This is exactly the behavior mean reversion describes.

I can also think about the spread directly. Let  $S = X - Y$  be the spread between the two log prices. Subtracting the two equations gives:

$$\frac{dS}{dt} = \frac{dX}{dt} - \frac{dY}{dt} \quad (3)$$

$$= -k(X - Y) - (-k(Y - X)) \quad (4)$$

$$= -2k(X - Y) \quad (5)$$

$$= -2kS \quad (6)$$

This is a simple exponential decay equation. The spread decays toward zero at rate  $2k$ . Clean, interpretable, and a reasonable first approximation of mean reversion in a correlated pair.

## 5. Methods: How Do We Actually Solve It?

I have my ODE system. Now I need to solve it. The analytical solution to  $\frac{dS}{dt} = -2kS$  is straightforward:

$$S(t) = S(0)e^{-2kt} \quad (7)$$

This is the eigenvalue solution and serves as one of my two approaches. The second is RK45, a numerical method I chose specifically because I wanted to explore alternatives to Euler's method and see how a more sophisticated solver would handle the system.

### What Is RK45?

RK45 is a numerical method for solving ODEs. To understand it, it helps to start with Euler's method, which is the simplest approach. Euler's method uses a single slope measurement at the start of each step to predict the next position:

$$y_{n+1} = y_n + h \cdot f(t_n, y_n) \quad (8)$$

Where  $h$  is the step size and  $f(t_n, y_n)$  is the slope at the current point. The problem is that using only one slope measurement leads to accumulated error, especially when the curve changes quickly.

RK45 improves on this by taking four slope measurements at each step and combining

them with weighted averaging:

$$k_1 = f(t, y) \tag{9}$$

$$k_2 = f\left(t + \frac{h}{2}, y + \frac{h \cdot k_1}{2}\right) \tag{10}$$

$$k_3 = f\left(t + \frac{h}{2}, y + \frac{h \cdot k_2}{2}\right) \tag{11}$$

$$k_4 = f(t + h, y + h \cdot k_3) \tag{12}$$

Where  $k_1$  is the slope at the start,  $k_2$  and  $k_3$  are slopes at two midpoint estimates, and  $k_4$  is the slope at the end. The final update combines all four:

$$y_{n+1} = y_n + \frac{h}{6}(k_1 + 2k_2 + 2k_3 + k_4) \tag{13}$$

The midpoint slopes  $k_2$  and  $k_3$  get double the weight because they describe the behavior of the solution across the step better than the endpoints alone. Smaller step sizes  $h$  always improve accuracy, at the cost of more computation.

### Why RK45 And Not Something Else?

RK45 is a natural choice here because it is robust enough to handle the extended system I develop later in the paper without modification. I used scipy's `solve_ivp` implementation with `method='RK45'`.

```

46 def solve_numerically(kappa, omega2, S0, M0, T):
47     t_span = (0, T - 1)
48     t_eval = np.arange(T, dtype=float)
49     sol = solve_ivp(ode_system(kappa, omega2), t_span, [S0, M0],
50                     t_eval=t_eval, method='RK45')
51     return sol, t_eval
52
53
54 def solve_analytically(kappa, omega2, S0, M0, t_eval):
55     A = np.array([[ -kappa, 1.0],
56                  [ -omega2, 0.0]])
57     eigenvalues, eigenvectors = np.linalg.eig(A)
58     c = np.linalg.solve(eigenvectors, np.array([S0, M0]))
59
60     analytical_SM = sum(
61         (c[i] * np.exp(eigenvalues[i] * t_eval) * eigenvectors[:, i:i+1]).real
62         for i in range(2)
63     )
64     return analytical_SM, eigenvalues
65

```

Figure 2: Python implementation of both solution methods. `solve_numerically` uses scipy's `solve_ivp` with `method='RK45'`, while `solve_analytically` computes the eigenvalue decomposition of the system matrix  $A$  and reconstructs the solution from eigenvectors and initial conditions.

## 6. Model Iterations: Did It Work? (Spoiler: Not At First)

This is where things get honest. The model I built in Section 4 is clean and interpretable on paper. What it does in practice is a different story. This section walks through three iterations of the model, what broke, what I changed, and what eventually started working.

### Iteration 1: The Original Model

The first model is the coupled ODE system from Section 4, solved with both the eigenvalue method and RK45, with  $k$  set to the Pearson correlation value of 0.9448. The result is shown in Figure 3.

Something was clearly wrong. The spread from my model was decaying to zero almost instantly while the actual spread was oscillating around a stable mean of roughly 0.05. The exponential decay equation  $S(t) = S(0)e^{-2kt}$  was doing exactly what it was supposed to do mathematically, the problem was that the real spread does not decay to zero. It mean

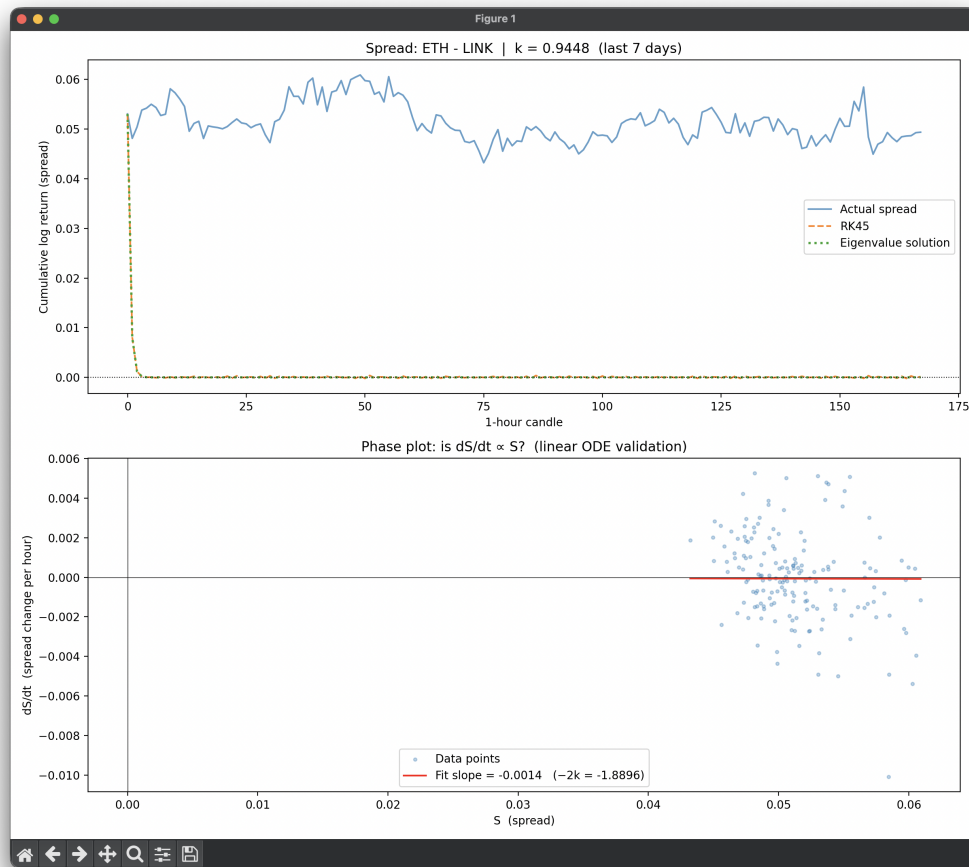


Figure 3: Iteration 1: Original model with  $k = 0.9448$ . Both the RK45 and eigenvalue solutions collapse to zero almost immediately while the actual ETH-LINK spread oscillates around a stable mean of roughly 0.05. The phase plot shows a nearly flat fit slope, confirming the linear assumption is weak at this parameter value.

reverts. Those are not the same thing.

I needed to add something to the model that captured the additional movement keeping the spread alive.

## Iteration 2: Adding The Market Factor

I introduced a new variable  $M$ , which I called the market factor, to account for the additional movement that the original model was missing. Looking at the actual spread, it was clearly oscillating around a mean rather than decaying, which made me think of harmonic oscillation. I chose an undamped oscillator for  $M$  because the market factor does not seem to decay over

time. The new system became:

$$\frac{dS}{dt} = k(S - \bar{S}) + M \quad (14)$$

$$\frac{dM}{dt} = \omega^2 S^* \quad (15)$$

Where  $\bar{S}$  is the spread mean,  $S^* = S - \bar{S}$  is the displacement of the spread from its mean, and  $\omega$  controls how strongly that displacement drives the market factor. Substituting and equating gives the second order equation:

$$\frac{d^2 S}{dt^2} - k \frac{d(S - \bar{S})}{dt} - \omega^2 S^* = 0 \quad (16)$$

I chose  $\omega^2$  deliberately to match the standard form of the harmonic oscillator equation, though any arbitrary constant would have worked the same way. The result is shown in Figure 4.

Progress. The model was now settling near the correct spread mean instead of collapsing to zero. But there was still no oscillation, and looking at the phase portrait it was clear something was still off. This is where I consulted Claude AI, which helped me realize I was using the wrong parameter values entirely.

### Iteration 3: Getting $k$ Right

The problem was with  $k$ . I had been using the Pearson correlation value of 0.9448 as  $k$ , but that was the wrong interpretation. In this model  $k$  should represent the mean reversion rate, specifically the rate at which the spread returns to its mean. That is a fundamentally different quantity from a correlation coefficient.

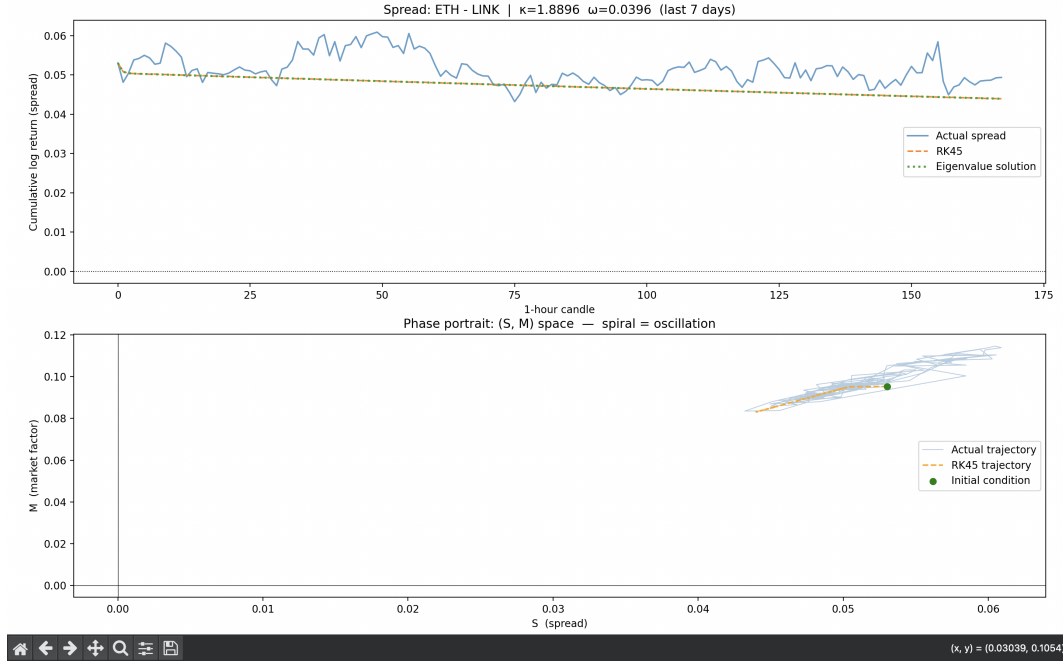


Figure 4: Iteration 2: Expanded model with market factor  $M$ ,  $\kappa = 1.8896$ ,  $\omega = 0.0396$ . The model now settles near the correct spread mean but there is still no oscillation. The phase portrait shows the trajectory concentrated in one region with no spiral structure.

Going back to the spread equation:

$$\frac{dS}{dt} = -2k(S^*) \quad (17)$$

The  $-2k$  term is the mean reversion rate. The factor of 2 comes from both assets contributing their own reversion rates.  $k$  should be estimated from the data directly, not borrowed from the correlation matrix. With the corrected parameter values the result is shown in Figure 5.

If you squint at the spread plot you can just about see the model trying to oscillate. But the real breakthrough here is the phase portrait. Compare it to the previous two iterations. There is now a clear underlying structure in the  $(S, M)$  space, a vector field with direction and form, even if the trajectory itself is still noisy. The model is starting to find something real.

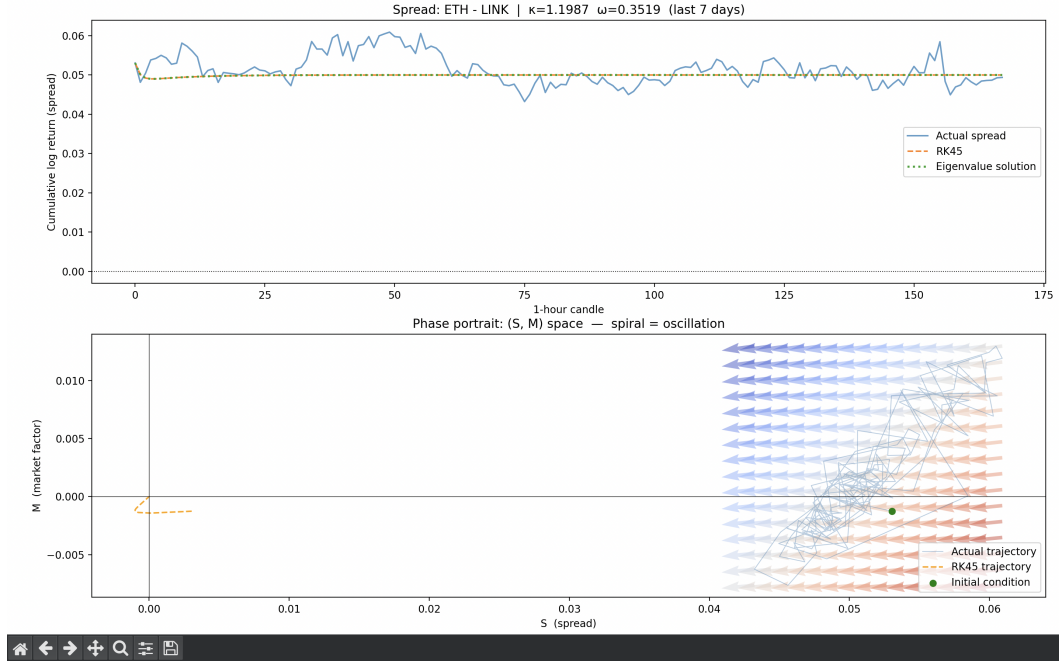


Figure 5: Iteration 3: Corrected mean reversion rate,  $\kappa = 1.1987$ ,  $\omega = 0.3519$ . The spread plot shows the model tracking close to the actual spread mean. More importantly the phase portrait now shows a clear vector field structure and the beginning of oscillatory behavior, confirming that the underlying dynamics are being captured even if imperfectly.

## 7. Results: So What Did I Actually Find?

Three iterations. Three different parameter regimes. Here is what they showed collectively.

### The Model Can Find The Mean

The most consistent result across all three iterations is that the model, once given the right parameters, settles near the correct spread mean. This is not trivial. The original exponential decay equation had no mechanism for mean reversion around a nonzero value. Adding the market factor  $M$  and correcting the interpretation of  $k$  gave the model that ability. By Iteration 3 the model was tracking the spread mean of approximately 0.05 with reasonable accuracy across 175 hourly candles.

## The Phase Portrait Tells The Real Story

The spread plots alone are somewhat misleading because they show how close the model is to the actual price, which is a high bar for any ODE applied to crypto data. The phase portraits in  $(S, M)$  space are more informative. Iteration 1 showed no structure. Iteration 2 showed the trajectory confined to a single region with no directional form. Iteration 3 showed a clear vector field with consistent direction, confirming that the system dynamics are being captured even when the trajectory itself is noisy.

## Noise Is The Remaining Problem

The model does not oscillate the way the actual spread does. The actual ETH-LINK spread moves constantly, driven by market forces that no deterministic ODE can fully capture.  $\omega$  in particular is difficult to estimate accurately because it requires computing a derivative of a derivative of already noisy data, compounding the noise at each step. A single value of  $\omega$  cannot describe the displacement dynamics across all 175 data points simultaneously. This is not a failure of the ODE framework but a signal that we should be looking elsewhere.

## What Would A Clean Oscillation Even Look Like?

For the curious reader, and I was very curious, I hardcoded parameter values of  $\kappa = 0.05$  and  $\omega = 0.20$  to see what the model would produce under ideal conditions. The result is shown in Figure 6.

This is what the model is capable of given the right parameters. The spiral in the phase portrait is not just visually satisfying, it is mathematically meaningful. An inward spiral in  $(S, M)$  space means the system is damping toward a fixed point, which is precisely what mean reversion looks like in a dynamical system. The challenge is not the model structure. The challenge is getting the parameters right from noisy real world data.

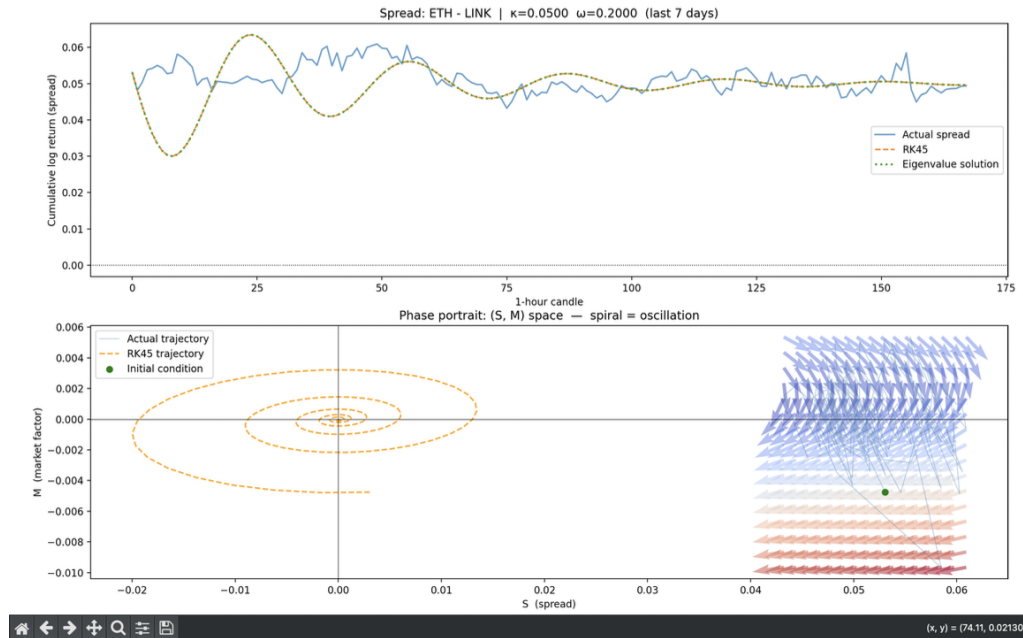


Figure 6: Hardcoded parameters  $\kappa = 0.05$ ,  $\omega = 0.20$ . The spread plot shows clear oscillation tracking the actual spread in the early candles before settling toward the mean. The phase portrait shows the characteristic inward spiral of a damped oscillator converging to a fixed point, exactly the structure the model is trying to find in the real data.

## 8. Discussion: Where Does This Go From Here?

The results of this project are promising but incomplete, and I think that is worth saying directly. The model finds the mean. The phase portrait shows structure. The curiosity plot shows the model is capable of oscillation under the right conditions. But the real data is noisy, the parameter estimation is imperfect, and the market does not sit still long enough for a deterministic ODE to fully catch up. Here is what I would change.

### The Pearson Assumption

I chose Pearson correlation because my ODE assumes a linear relationship between the two assets. That is a reasonable justification but it is still a simplifying assumption. The cleaner approach would be to plot  $\frac{dS}{dt}$  against  $S$  directly and check whether the relationship is actually linear before committing to a linear ODE. A curved relationship would push toward Spearman and potentially a nonlinear system. This is the first thing I would revisit.

## The Unforced Oscillator

I used an undamped unforced oscillator for the market factor  $M$ . The reasoning was that the market factor does not seem to decay over time, which is true, but an unforced oscillator has no external driving term. Real markets are constantly being driven by external forces, news, liquidity, sentiment, macroeconomic events. A forced harmonic oscillator would be a more realistic model for  $M$ , though figuring out what the forcing term should actually be is a genuinely hard problem I do not have an answer to yet.

## Noise

The biggest limitation of the current model is that it has no mechanism for noise. Every ODE in this paper is deterministic. Given the same initial conditions and parameters it will always produce the same output. Real financial data does not work that way. The natural next step appears to be a stochastic differential equation. From my research, the Ornstein-Uhlenbeck process seems like the right first candidate to explore in that direction. However before I dive into stochastic territory I want to exhaust what is possible within the deterministic ODE framework first. There is still a lot of room to improve parameter estimation, test nonlinear systems, and experiment with forced oscillator models before the math requires randomness built in.

## A Note On Scope

This project did not start as a class assignment. I have been working on tracking and modeling cryptocurrency movements since the beginning of the semester, mostly through the lens of linear algebra. The correlation matrix, the pair selection method, and much of the underlying code were already built before this project was assigned. Taking Differential Equations 230 felt like the missing piece I did not know I needed. The tools I was learning in class were exactly what my existing work was pointing toward, I just had not had the language for it yet. The class project was not a new direction. It was a natural extension of

something I had already been building.

What that means for the model is that the infrastructure is solid even where the mathematics is still developing. The data pipeline, the pair selection, the numerical solver, all of it was already in place. Differential equations gave me the framework to ask better questions about what the data was doing. I am genuinely excited to keep working on this over the summer. The math is not finished. It is just getting started.

## Appendix: Use of Large Language Models

I used Claude AI (Anthropic) in two ways during this project. The first was during the modeling process itself, where it helped me identify that I was using the wrong parameter values for  $k$ , specifically that I was using the Pearson correlation coefficient where I should have been using the mean reversion rate estimated from the data directly. That conversation is referenced in Section 6.

The second was in preparing this document. I had no prior experience with LATEX and used Claude AI to convert my written draft into a properly formatted LATEX script, including typesetting all mathematical equations, placing and labeling figures, and structuring the document to meet the formatting requirements of this assignment. The ideas, analysis, code, and conclusions are entirely my own. Claude AI handled the presentation layer.

## References

- [1] MIT OpenCourseWare. *MIT 18.03 Differential Equations, Spring 2006*. YouTube Playlist, 33 videos. <https://www.youtube.com/playlist?list=PLEC88901EBADDD980>
- [2] One Minute Economics. *Market Makers (Liquidity Providers) and the Bid-Ask Spread Explained in One Minute*. YouTube, Feb 13, 2020. <https://www.youtube.com/watch?v=1W01GmrI1S0>

- [3] Brenndoerfer, M. *Differential Calculus, Optimization, and Quantitative Finance*. <https://mbrenndoerfer.com/writing/differential-calculus-optimization-quantitative-finance>